

Wasserstandsregelung mit optischer Füllhöhenerkennung

Der Aufbau, die Bildauswertung, der PI-Regler und das Behältermodell aus der
gemessenen Sprungantwort

Prof. Dr.-Ing. Ralph Wystup M.Sc. — erstellt mit KI und Agent (Claude Code, Anthropic)

Fassung 1.0 · 29.09.2026

Inhaltsverzeichnis

1 Worum es geht	2
2 Der Aufbau und die realen Schnittstellen	2
2.1 Die serielle Schnittstelle zum EasyPort	2
2.2 Die belegten Register	3
2.3 Was die Messreihe über den Wandler verrät	4
2.4 Die Kamera	4
3 Die Bildauswertung, Schritt für Schritt	5
3.1 Der Bildausschnitt (ROI)	6
3.2 Methode „Helligkeit“ (Voreinstellung)	6
3.3 Methode „Canny“	7
3.4 Methode „Farbfilter (HSV)“	7
3.5 Was mit dem Rohergebnis geschieht	7
3.6 Kalibrierung: Bildpunkt → Millimeter	8
4 Der Regler	8
4.1 Takt und Zeitschritt	8
4.2 Die Gleichung	8
4.3 Anti-Windup und Sollwertsprung	8
4.4 Was gemessen und angezeigt wird	9
5 Das Streckenmodell des Behälters	9
5.1 Die Bilanz	9
5.2 Die Messung	9
5.3 Die Anpassung	10
5.4 Die Kennwerte des Behälters	11
6 Der geschlossene Kreis	11
6.1 Übertragungsfunktionen	11
6.2 Kennwerte und Stabilität	12
6.3 Störübertragungsfunktion	13
6.4 Zwei unabhängige Wege zu denselben Kennwerten	13
7 Bedienung	15
7.1 Das Programm am PC	15
7.2 Die Kamera einstellen	15
7.3 Kalibrieren	15

7.4 Aufzeichnen	16
8 Die Seite im Browser	16
9 Was offen ist	17
10Anhang: die Dateien	17

1 Worum es geht

Ein durchsichtiger Behälter wird von einer Pumpe gefüllt und läuft über einen Rücklauf wieder leer. Ein Ultraschallsensor misst die Füllhöhe; ein PI-Regler auf einem PC stellt die Pumpe. Ein Ablassventil öffnet auf Knopfdruck und erzeugt eine Störung. Parallel dazu blickt eine ESP32-CAM auf denselben Behälter, und das Programm sucht **im Bild** die Wasserlinie — das ist die optische Füllhöhenerkennung des Titels.

Dieses Blatt beschreibt genau das Programm `Wasserstandsregelung_Bild_6.py`, nicht ein gedachtes Verfahren. Jede Zahl, jede Schwelle und jede Formel ist entweder aus dem Programm entnommen (mit Angabe der Funktion), aus der Messreihe `Daten/sprungantwort_2026-06-25.csv` gerechnet oder in `Modell/streckenmodell.py` und `Modell/nachrechnung.py` nachvollziehbar hergeleitet. Wo etwas **nicht** belegbar ist, steht das ausdrücklich da (Abschnitt „Was offen ist“); geschätzt wird nichts.

Eine Besonderheit ist gleich vorweg festzuhalten, weil sie das Verständnis der ganzen Anlage prägt:

Die Bildauswertung greift nicht in die Regelung ein. Sie läuft parallel und zeigt ihren Wert an. Geregelt wird ausschließlich mit dem Ultraschallsensor. Im Programm steht das als Kommentar über der Bildverarbeitung: *„Ergebnis wird NUR angezeigt, beeinflusst die Regelung NICHT.“*

Die Kamera ist damit ein **zweiter, unabhängiger Messweg** auf dieselbe Größe — genau das, was Grundsatz 1 der Werkstatt verlangt. Sie ist nicht Ersatz für den Sensor, sondern seine Probe.

2 Der Aufbau und die realen Schnittstellen

Es gibt drei Geräte und zwei Schnittstellen:

1. die **Füllstandsanlage** (Behälter, Pumpe, Ablassventil, Ultraschallsensor, Durchfluss- und Drucksensor, zwei Schwimmerschalter) auf 24-V-Klemmen,
2. den **EasyPort** als Umsetzer zwischen diesen Klemmen und dem PC, über USB als serielle Schnittstelle,
3. den **PC** mit dem Python-Programm (Tkinter für die Oberfläche, OpenCV für das Bild, Matplotlib für die Verläufe),
4. die **ESP32-CAM**, die über WLAN einen MJPEG-Strom liefert.

2.1 Die serielle Schnittstelle zum EasyPort

Das Programm öffnet die Schnittstelle mit festen Werten (Funktion `serial.Serial` im Kopf):

Größe	Wert
Schnittstelle	COM9
Übertragungsrate	115200 Bd
Rahmen	8 Datenbits, keine Parität, 1 Stoppbit
Wartezeit lesen / schreiben	je 0,5 s

Der Aufbau mit den realen Schnittstellen: Anlage — EasyPort — PC-Programm — ESP32-CAM

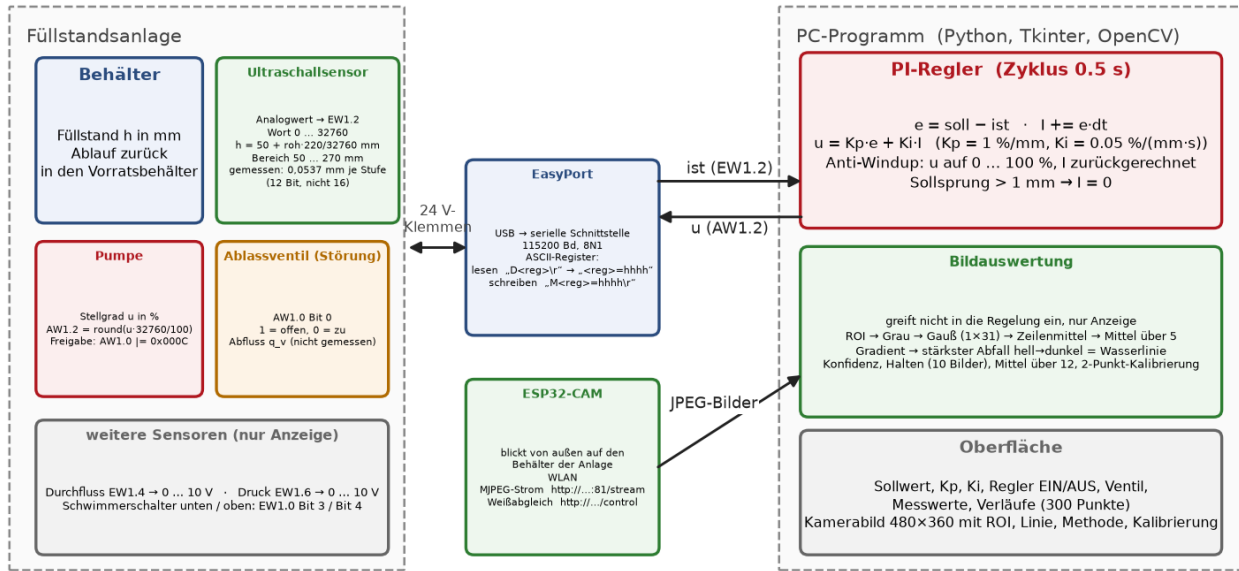


Abbildung 1: Der Aufbau mit den realen Schnittstellen

Der EasyPort spricht ein **ASCII-Registerprotokoll**, kein Modbus. Das Programm benutzt genau zwei Befehle (read_register, write_register):

- **Lesen:** D<Register>\r senden, bis zum Wagenrücklauf lesen, die Antwort hat die Form <Register>=hhhh. Das Programm trennt an = und liest den Wert als **Hexadezimalzahl** (int(value, 16)).
- **Schreiben:** M<Register>=hhhh\r senden; die Antwort wird gelesen und verworfen.

Beide Zugriffe laufen unter einem threading.Lock, weil zwei Abläufe gleichzeitig auf die Schnittstelle zugreifen (Regelschleife und Bedienknöpfe). Vor jedem Lesen wird der Empfangspuffer verworfen (reset_input_buffer) — sonst käme die Antwort eines früheren Befehls zurück.

2.2 Die belegten Register

Das ist die vollständige Liste der Register, die im Programm vorkommen — mehr sind es nicht:

Register	Richtung	Bedeutung	Umrechnung im Programm
EW1.0, Bit 3	lesen	Schwimmerschalter unten	<code>fs_unten(): (roh & (1<<3)) != 0</code>
EW1.0, Bit 4	lesen	Schwimmerschalter oben	<code>fs_oben(): (roh & (1<<4)) != 0</code>
EW1.2	lesen	Füllhöhe (Ultraschall)	<code>level_mm():</code> $h = 50,0 + roh \cdot \frac{220,0}{32760}$ mm
EW1.4	lesen	Durchfluss	<code>durchfluss_volt():</code> $U = roh \cdot \frac{10,0}{32760}$ V

Register	Richtung	Bedeutung	Umrechnung im Programm
EW1.6	lesen	Druck	<code>druck_volt()</code> : $U = \text{roh} \cdot \frac{10,0}{32760} \text{ V}$
AW1.0, Bit 0	schreiben	Ablassventil	1 = offen, 0 = geschlossen
AW1.0, Bits 2 und 3	schreiben	Pumpenfreigabe	<code>pump_set</code> setzt AW1.0 <code> = 0x000C</code>
AW1.2	schreiben	Pumpenstellgrad	<code>pump_set: roh =</code> <code>round($u \cdot \frac{32760}{100}$)</code>

Bemerkenswert an `pump_set`: die Freigabebits werden **vor jedem** Stellwert neu gesetzt, weil das Register AW1.0 auch das Ventilbit trägt und sonst beim Ventilschalten verloren ginge. `pump_stop` schreibt erst den Stellwert 0 und nimmt danach die Freigabe zurück (AW1.0 &= 0xFFF3).

Durchfluss, Druck und die beiden Schwimmerschalter werden nur **angezeigt**. Sie gehen nirgends in die Regelung ein; ihre Umrechnung in physikalische Einheiten (l/min, bar) steht nicht im Programm, darum steht sie auch hier nicht.

2.3 Was die Messreihe über den Wandler verrät

Die Formel `level_mm()` rechnet mit der vollen Spanne 0 ... 32760, also mit einem 16-Bit-Wort. Ob der Analogeingang wirklich so fein auflöst, steht in der Messreihe selbst. Rechnet man die 370 aufgezeichneten Füllhöhen in Rohwerte zurück, so gilt für **alle** 370 Werte:

$$\text{roh} = 8k + 5, \quad k \in \mathbb{N}.$$

Der größte gemeinsame Teiler aller auftretenden Sprünge ist 8. Der Wandler liefert also $32760/8 = 4095$ Stufen — **12 Bit, nicht 16**. Daraus folgt für die Rückführung:

Größe	Wert
Rechenschritt der Formel ($220/32760$)	0,00672 mm
tatsächlicher Messschritt ($8 \cdot 220/32760$)	0,05372 mm
fester Versatz ($5 \cdot 220/32760$)	0,03358 mm
Stellschritt der Pumpe ($100/32760$)	0,00305 %

Der feste Versatz von 0,034 mm verschwindet in jeder Kalibrierung und ist ohne Bedeutung; die Auflösung von 0,054 mm ist die Zahl, mit der zu rechnen ist. Sie ist um mehr als zwei Größenordnungen feiner als jede Regelabweichung, die später auftritt — die Quantisierung ist also **nicht** die begrenzende Größe. Gerechnet in `Modell/nachrechnung.py`, Abschnitt 5.

Nicht belegt: Die Messgenauigkeit des Ultraschallsensors selbst (Linearität, Temperaturgang, Streuung bei bewegter Oberfläche) ist nicht gemessen worden. 0,054 mm ist die Stufe des Wandlers, nicht die Genauigkeit der Messung.

2.4 Die Kamera

Die ESP32-CAM liefert einen MJPEG-Strom. Das Programm holt ihn in einem eigenen Faden (`mjpeg_thread`):

Größe	Wert im Programm
Bildstrom	http://<IP-der-Kamera>:81/stream
Einstellungen	http://<IP-der-Kamera>/control?var=...&val=...
Anzeigegröße	480 × 360 Bildpunkte (CAM_W, CAM_H)
Wiederverbindung	alle 3 s (RECONNECT_INTERVAL)
als offline gilt	5 s ohne neues Bild (TIMEOUT_OFFLINE)
Bildtakt der Anzeige	40 ms (root.after(40, update_kamera))

Der Strom wird **nicht** von einer Bibliothek zerlegt, sondern von Hand: das Programm sammelt Bytes in einem Puffer und sucht darin die JPEG-Marken FF D8 (Bildanfang) und FF D9 (Bildende). Was dazwischen liegt, geht an cv2.imdecode. Wächst der Puffer über 100 000 Bytes, ohne dass ein vollständiges Bild darin steht, wird er verworfen — so kann ein abgerissener Strom die Schleife nicht verstopfen. **Jedes** dekodierte Bild geht unmittelbar in bv_verarbeite, also läuft die Bildauswertung mit der Bildrate der Kamera und nicht mit dem Reglertakt.

Der Weißabgleich ist über fünf Stufen umschaltbar (WB_MODI: Auto, Sonnig, Bewölkt, Büro, Wohnraum). „Auto“ schaltet awb=1 und wb_mode=0, jede feste Stufe schaltet awb=0 und wb_mode=<Nummer>. Die Anforderungen laufen in einem eigenen Faden, damit die Oberfläche nicht stehenbleibt, wenn die Kamera nicht antwortet.

Nicht belegt: Brennweite, Bildwinkel, Abstand zum Behälter und Auflösung des Sensors stehen nicht im Programm. Die Umrechnung Bildpunkt → Millimeter wird deshalb nicht aus der Geometrie berechnet, sondern gemessen (Abschnitt „Kalibrierung“).

3 Die Bildauswertung, Schritt für Schritt

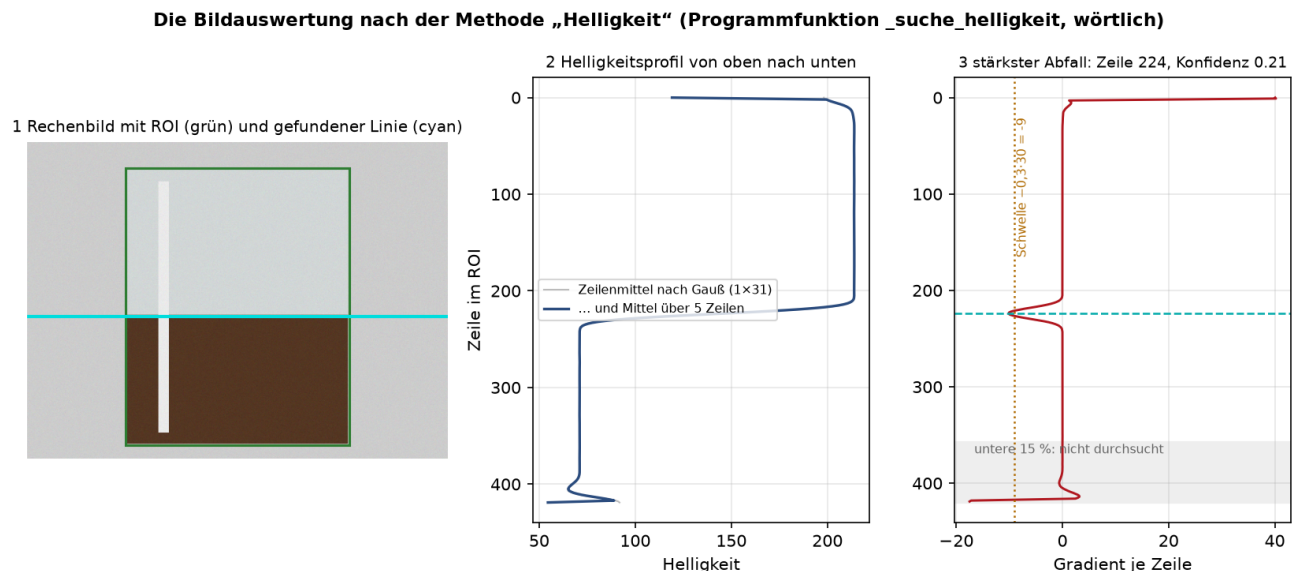


Abbildung 2: Die Bildauswertung nach der Methode „Helligkeit“

Die Bildauswertung liegt im Programm zwischen den Zeilen 327 und 508. Diese Zeilen sind **wörtlich** nach Modell/bv_referenz.py herausgeschnitten (durch Modell/erstelle_bv_referenz.py), damit dieselbe Erkennung ohne Kamera, ohne EasyPort und ohne Oberfläche auf beliebigen Bildern läuft. Alles, was hier steht, ist an diesem Auszug nachprüfbar.

3.1 Der Bildausschnitt (ROI)

Ausgewertet wird nicht das ganze Bild, sondern ein Rechteck $\text{roi} = [\text{x1}, \text{y1}, \text{x2}, \text{y2}]$ im **Originalbild** (nicht in der 480×360 -Anzeige). Voreinstellung: $[50, 30, 590, 450]$. Der Bediener zieht es mit der Maus über dem Kamerabild neu auf; `maus_release` rechnet die Anzeigekoordinaten mit den Faktoren $s_x = w_{\text{orig}}/480$ und $s_y = h_{\text{orig}}/360$ auf das Originalbild zurück und verlangt mindestens 10 Bildpunkte Kantenlänge. Bei jeder Änderung wird der Glättungspuffer geleert.

In `bv_verarbeite` wird das Rechteck auf das Bild beschnitten; ist der Ausschnitt in einer Richtung schmaler als 10 Bildpunkte, bricht die Auswertung ohne Ergebnis ab.

3.2 Methode „Helligkeit“ (Voreinstellung)

Das ist die Methode für trübes Wasser in einem durchsichtigen Behälter: **die Luft über dem Wasser ist heller als das Wasser**. Die Wasserlinie ist damit die Zeile mit dem stärksten Helligkeitsabfall von oben nach unten. Sieben Schritte, in dieser Reihenfolge (`_suche_helligkeit`):

1. **Graustufen:** `cv2.cvtColor(ausschnitt, cv2.COLOR_BGR2GRAY)`, also die OpenCV-Gewichtung $g = 0,299 R + 0,587 G + 0,114 B$.
2. **Senkrechte Glättung:** `cv2.GaussianBlur(grau, (1, 31), 0)`. Die Kerngröße ist (Breite = 1, Höhe = 31) — es wird **nur in Spaltenrichtung** geglättet, über 31 Zeilen. Das unterdrückt Reflexe und Rauschen und verschmiert zugleich die Kante über rund 31 Zeilen; das ist der Preis der Robustheit und die Ursache der Empfindlichkeitsschwelle in Schritt 6.
3. **Zeilenmittel:** `grau.mean(axis=1)` ergibt das Helligkeitsprofil $p(i)$ über die Zeilen i des Ausschnitts.
4. **Gleitendes Mittel über 5 Zeilen:** `np.convolve(p, np.ones(5)/5, mode='same')`.
5. **Ableitung:** `np.gradient(p)` — der zentrale Differenzenquotient, also $g(i) = \frac{1}{2}(p(i+1) - p(i-1))$ im Inneren und einseitig an den Rändern.
6. **Suche:** durchsucht wird nur der obere Teil, `such_bis = int(n*0,85)`; der untere Rand ist Behälterboden und Tisch. Der stärkste Abfall ist $\min g$. Ist dieser Wert **größer** als $-0,3 \cdot \text{hell_schwelle}$, gilt kein Übergang als gefunden und die Methode liefert nichts. Mit der Voreinstellung `hell_schwelle = 30` ist die Schranke -9 Graustufen je Zeile. Die gesuchte Zeile ist `np.argmin`. *Anmerkung:* Der Kommentar im Programm spricht von „oberen 80 %“, der Code rechnet mit 0,85. Es gilt der Code: 85 %. *Folge für den Aufbau:* Was im untersten Sechstel des Bildausschnitts liegt, findet diese Methode nicht. Zieht der Bediener den ROI eng um den Glaseinhalt — wie die Bedienhilfe im Programm es rät —, so ist das unterste Sechstel des Füllbereichs für die Kamera unsichtbar. In der Browser-Seite ist diese Grenze aus der Zeichnung gerechnet und im Versuch bestätigt: sie liegt dort bei 80,4 mm; 12 mm darunter findet die Methode nichts, 12 mm darüber liest sie 93,0 mm.
7. **Konfidenz:** mit $\Delta p = \max p - \min p$ (ist $\Delta p < 5$, gilt das Bild als flau und es gibt kein Ergebnis)

$$\text{konf} = \min\left(1, 0, \frac{|\min g|}{0,3 \Delta p}\right).$$

Die Konfidenz vergleicht also den Abfall **je Zeile** mit 30 % des Helligkeitsumfangs im ganzen Ausschnitt.

Was Schritt 2 für Schritt 6 bedeutet, lässt sich geschlossen angeben und von Hand nachrechnen: ein Sprung um Δ Graustufen wird von einem Gauß mit $\sigma \approx 31/6 \approx 5,2$ Zeilen zu einer Flanke mit dem größten Betrag

$$|g|_{\max} \approx \frac{\Delta}{\sigma\sqrt{2\pi}} \approx \frac{\Delta}{13}.$$

Damit die Voreinstellung anspricht ($|g|_{\max} \geq 9$), braucht es einen Helligkeitsunterschied von rund **117 Graustufen** zwischen Luft und Wasser. Das ist keine Theorie: das Rechenbild des Bildes

oben ist genau daran geeicht. Mit einem Wasser-Grauwert von 88 (Unterschied 126) erreicht der Gradient $-8,94$ und die Methode findet **nichts**; mit dem gezeichneten Grauwert 62 (Unterschied 152) erreicht er $-10,9$ und sie findet die Linie. Wer in der Anlage trübes Wasser bei schwacher Beleuchtung hat, muss die Empfindlichkeit heruntersetzen — der Schieber reicht von 5 bis 80.

3.3 Methode „Canny“

`_suche_wasserlinie_canny` sucht **waagerecht durchgehende Kanten**:

1. Kerngröße ungerade machen (`blur_kern`, Voreinstellung 5; gerade Werte werden um 1 erhöht),
2. Graustufen, `cv2.GaussianBlur(grau, (k, k), 0)` — hier quadratisch, also in beide Richtungen,
3. `cv2.Canny(grau, canny_t1, canny_t2)` mit den Voreinstellungen 30 und 80,
4. je Zeile den Anteil der Kantenpunkte bilden: $a(i) = \frac{1}{255b} \sum_j \text{kanten}(i, j)$ mit der ROI-Breite b ,
5. Kandidaten sind die Zeilen mit $a(i) > 0,06$; gewählt wird die Zeile mit dem **größten** Anteil,
6. Konfidenz = $\min(1,0, a/0,4)$ — eine Zeile, die zu 40 % aus Kantenpunkten besteht, gilt als voll vertrauenswürdig.

Schieber: T1 von 5 bis 150, T2 von 10 bis 300, Weichzeichnung von 1 bis 21.

3.4 Methode „Farbfilter (HSV)“

`_suche_wasserlinie_farbe` sucht die **Oberkante einer Farbmaske**:

1. `cv2.cvtColor(..., cv2.COLOR_BGR2HSV)`; OpenCV zählt den Farbton von 0 bis 179,
2. `cv2.inRange(hsv, (h_lo, s_lo, v_lo), (h_hi, 255, 220))` mit den Voreinstellungen $h \in [10, 30]$ (rot-braun bis orange), $s \geq 40$, $v \in [40, 220]$ — die Obergrenze 220 für den Hellwert schließt Glanzlichter aus,
3. Lücken schließen: `MORPH_CLOSE` mit einem Rechteck 5×3 (breiter als hoch, weil die Wasserfläche waagerecht zusammenhängt),
4. je Zeile die Maskensumme; als besetzt gilt eine Zeile ab $0,12 \cdot 255 \cdot b$, also ab 12 % der Breite,
5. genommen wird die **oberste** besetzte Zeile (`kandidaten.min()`) — anders als bei Canny, wo die stärkste genommen wird,
6. Konfidenz = $\min\left(1,0, \frac{\text{Summe}}{255b \cdot 0,5}\right)$: eine Zeile, die zur Hälfte in der Maske liegt, gilt als voll vertrauenswürdig.

3.5 Was mit dem Rohergebnis geschieht

`bv_verarbeite` nimmt Zeile und Konfidenz der gewählten Methode entgegen und macht daraus den angezeigten Wert:

1. **Annahme oder Ablehnung.** Liegt kein Ergebnis vor oder ist die Konfidenz kleiner als `konf_schwelle` (Voreinstellung 0,05, Schieber 0,01 bis 0,30), so wird der Wert **gehalten**: `bv_hold_count` zählt hoch, der letzte gültige Wert bleibt stehen. Erst nach `HOLD_MAX = 10` gehaltenen Bildern (Schieber 1 bis 60) wird die Anzeige gelöscht. Ein gültiges Ergebnis setzt den Zähler auf 0 zurück.
2. **Zurück ins Originalbild:** `beste_orig = y1 + beste_roi`.
3. **Mittelung über 12 Bilder:** ein Ringpuffer der Länge `GLAETT_N = 12`; der angezeigte Bildpunkt ist `int(np.mean(puffer))` — also der **abgeschnittene**, nicht der gerundete Mittelwert. Das ist ein systematischer Versatz von bis zu einem Bildpunkt nach oben.
4. **Umrechnung in Millimeter** (Abschnitt „Kalibrierung“).

Die Farbe der eingblendeten Linie sagt, in welchem dieser Zustände die Auswertung ist: Cyan bei Konfidenz über 0,15, Blau darunter, **Orange**, solange gehalten wird. Das ist eine Bedienregel, keine Verzierung — eine orange Linie heißt: die Kamera sieht gerade nichts, der Wert ist alt.

3.6 Kalibrierung: Bildpunkt → Millimeter

Es gibt **keine** Umrechnung aus der Geometrie. Die Umrechnung ist eine Gerade durch zwei gemessene Punkte (pixel_zu_mm):

$$h(p) = m_1 + (p - p_1) \frac{m_2 - m_1}{p_2 - p_1}.$$

Der Bediener bringt den Füllstand auf einen bekannten Wert, trägt ihn in das Feld ein und drückt „Punkt setzen“; das Programm merkt sich das Paar (erkannter Bildpunkt, Millimeterwert). Beim zweiten Punkt gilt die Kalibrierung. Ein dritter Punkt verdrängt den ersten (kalib_punkte.pop(0)), es sind also immer die **beiden letzten**. Ohne zwei Punkte zeigt das Programm „— (nicht kalibriert)“ an; die Linie wird trotzdem gezeichnet.

Die Gerade unterstellt, dass die Kamera senkrecht auf den Behälter blickt und der Behälter über die Messhöhe denselben Querschnitt hat. Beides ist nicht gemessen — siehe „Was offen ist“.

4 Der Regler

Der Regler ist ein **PI-Regler mit Anti-Windup**, geschrieben in regulation(). Er läuft in der Tkinter-Ereignisschleife, nicht in einem eigenen Faden.

4.1 Takt und Zeitschritt

root.after(500, regulation) am Ende jedes Durchlaufs: der **Nennzyklus ist 0,5 s**. Der tatsächlich verwendete Zeitschritt wird aber gemessen und begrenzt:

```
dt = now - last_time
dt = max(0.001, min(dt, 2.0))
```

Das ist wichtig und keine Kosmetik: root.after garantiert nur ein Mindestintervall. Bleibt die Oberfläche hängen (Zeichnen der Verläufe, Kamerabild), so wäre der Zeitschritt größer; die Klammerung auf höchstens 2 s verhindert, dass ein einziger langer Durchlauf das Integral aufbläht. Der erste Durchlauf setzt nur last_time und rechnet nicht.

4.2 Die Gleichung

Mit dem Sollwert w (Feld „Sollwert“), dem Istwert $y = \text{\$ level_mm}()$ und den Beiwerten K_p und K_i aus den Eingabefeldern:

$$e = w - y, \quad I \leftarrow I + e \cdot \Delta t, \quad u = K_p e + K_i I.$$

Das ist die **Rechteckregel von links** (Euler vorwärts) für das Integral — von Hand nachrechenbar, und genau so steht es im Programm.

Voreinstellungen der Oberfläche: $w = 150$ mm, $K_p = 1,0$ %/mm, $K_i = 0,05$ %/(mm·s).

4.3 Anti-Windup und Sollwertsprung

Die Stellgröße ist auf 0 ... 100 % begrenzt. Bei Anschlag wird das Integral **zurückgerechnet**, so dass es genau den Wert hat, der zur Grenze führt:

```
if output > 100.0: integral -= (output - 100.0) / ki ; output = 100.0
elif output < 0.0: integral -= output / ki ; output = 0.0
```

Man rechnet das in einer Zeile nach: nach der Korrektur ist $K_p e + K_i I = 100$ bzw. $= 0$. Das Integral „lädt sich“ also nicht auf, solange die Pumpe am Anschlag steht — und der Regler löst sich sofort vom Anschlag, sobald die Regelabweichung das Vorzeichen wechselt. (Bei $K_i = 0$ setzt das Programm einen Ersatzwert 10^{-9} ein, um die Division zu vermeiden.)

Zweite Vorkehrung: ändert der Bediener den Sollwert um **mehr als 1,0 mm**, wird das Integral auf 0 gesetzt. Der Regler beginnt den Sprung also aus dem Stand, nicht mit dem alten Ladezustand.

Der Knopf „Regler EIN/AUS“ schaltet die Rechnung; beim Einschalten werden Integral, Zeitmarke und letzter Sollwert zurückgesetzt, beim Ausschalten wird die Pumpe angehalten. „Integral zurücksetzen“ nullt nur das Integral, „Pumpe STOP“ schreibt Stellwert 0 und nimmt die Freigabe.

4.4 Was gemessen und angezeigt wird

Jeder Durchlauf liest zusätzlich Durchfluss, Druck und die beiden Schwimmerschalter und schreibt Istwert, Sollwert und Stellgröße in drei Ringpuffer mit **300 Punkten** (MAX_POINTS); bei 0,5 s Zyklus sind das 150 s Verlauf. Die beiden Diagramme werden in jedem Zyklus vollständig neu gezeichnet. Eine Ausnahme in `regulation()` bricht den Zyklus nicht ab: sie wird im Fehlerfeld angezeigt, und der nächste Durchlauf wird trotzdem angemeldet.

5 Das Streckenmodell des Behälters

5.1 Die Bilanz

Für einen Behälter mit dem lichten Querschnitt A gilt die Mengenbilanz

$$A \frac{dh}{dt} = q_{zu} - q_{ab}.$$

Der Zufluss ist dem Stellgrad verhältnissgleich, $q_{zu} = k_p u$. Für den Abfluss über den Rücklauf gibt es zwei Ansätze, die sich an der Messung unterscheiden lassen:

(a) verhältnissgleich zur Höhe (laminarer Widerstand, enge Drossel): $q_{ab} = k_a (h - h_0)$. Damit

$$\frac{A}{k_a} \frac{dh}{dt} + (h - h_0) = \frac{k_p}{k_a} u \quad \Longleftrightarrow \quad T \dot{h} + (h - h_0) = K_u u,$$

also ein **PT1-Glied** mit $T = A/k_a$ und $K_u = k_p/k_a$.

(b) nach Torricelli (freier Ausfluss aus einer Öffnung): $q_{ab} = \mu A_0 \sqrt{2g(h - h_b)}$, also

$$\dot{h} = a - c \sqrt{h - h_b}.$$

Dazu kommt in beiden Fällen eine **Totzeit** T_d : die Pumpe fördert durch eine Leitung, und der Ultraschallsensor liefert seinen Wert nicht sofort.

5.2 Die Messung

Am 25.06.2026 wurde die Pumpe bei $t = 0$ von 0 auf 100 % gestellt und der Füllstand aufgezeichnet, bis 200 mm erreicht waren:

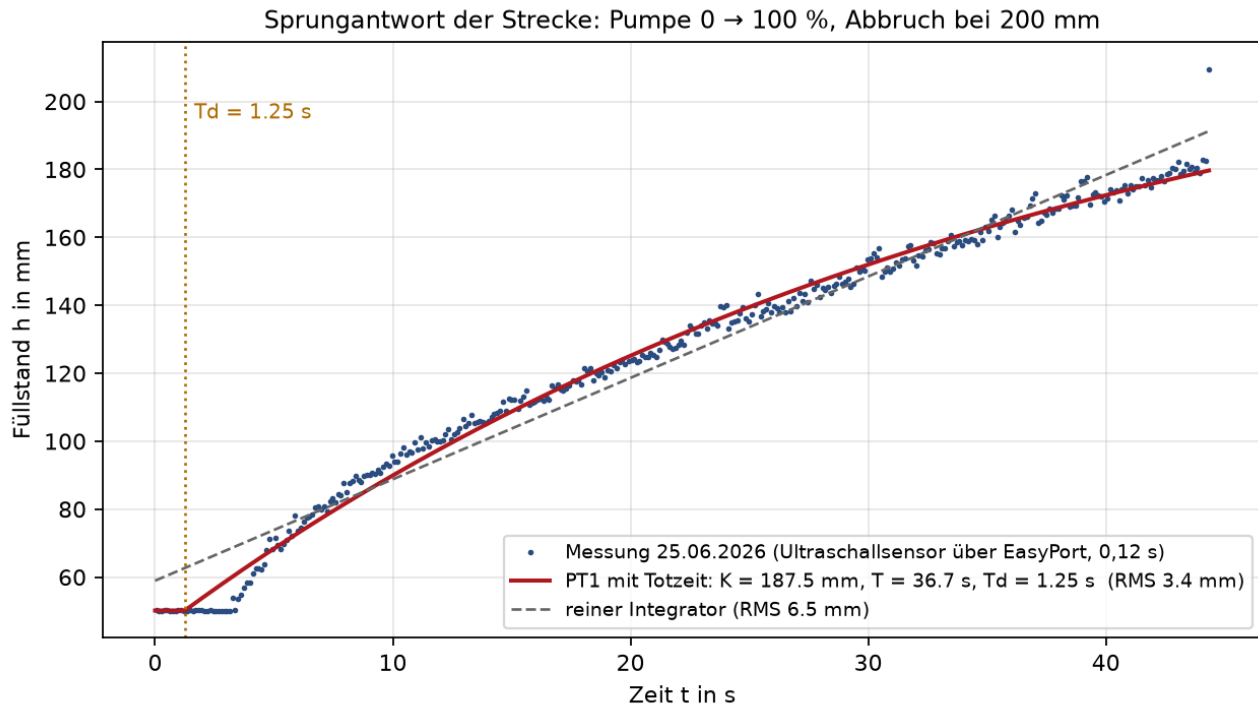


Abbildung 3: Sprungantwort der Strecke

Größe	Wert
Punkte	370
mittlerer Abstand	0,12 s
Dauer	44,34 s
Anfang	50,25 mm
Ende	209,59 mm
Rauschen (Standardabweichung der ersten 20 Punkte)	0,062 mm

Zwei Dinge sind an dieser Messung festzuhalten, bevor irgendetwas angepasst wird:

1. **Sie ist kürzer als eine Zeitkonstante mal 1,3.** Bei $T = 36,7$ s deckt sie 1,21 T ab; der Endwert wird also **hochgerechnet**, nicht gemessen. Die Unsicherheit dieser Hochrechnung ist beziffert (unten, σ_K).
2. **Der letzte Punkt ist ein Ausreißer.** Er springt um 27,13 mm gegenüber seinem Vorgänger, bei 0,12 s Abstand — das kann kein Füllstand sein. Er stammt aus dem Abbruch der Aufzeichnung. Lässt man ihn weg, so ergibt die Anpassung $K = 183,59$ mm statt 187,48 mm und $T = 35,51$ s statt 36,69 s. Beide Änderungen liegen **innerhalb einer Standardabweichung** ($\sigma_K = 3,79$ mm, $\sigma_T = 1,23$ s). Der Ausreißer wird darum nicht entfernt, sondern beziffert.

5.3 Die Anpassung

Modell/streckenmodell.py passt drei Modelle an dieselbe Messreihe an (kleinste Fehlerquadrate, scipy.optimize.curve_fit bzw. least_squares):

Modell	Parameter	Restfehler (RMS)
PT1 mit Totzeit	$K = 187,48 \text{ mm}, T = 36,69 \text{ s},$ $T_d = 1,252 \text{ s}, h_0 = 50,16 \text{ mm}$	3,42 mm
Behälter nach Torricelli	$a = 8,452 \text{ mm/s}, c = 0,6173,$ $T_d = 3,26 \text{ s}, h_b = 50,0 \text{ mm}$	3,90 mm
reiner Integrator (kein Abfluss)	$a = 2,985 \text{ mm/s}, T_d = -3,85 \text{ s}$	6,55 mm

Standardabweichungen der PT1-Anpassung: $\sigma_K = 3,79 \text{ mm}, \sigma_T = 1,23 \text{ s}, \sigma_{T_d} = 0,237 \text{ s}, \sigma_{h_0} = 1,04 \text{ mm}.$

Drei Schlüsse:

- Der reine Integrator scheidet aus: sein Restfehler ist fast doppelt so groß, und seine Totzeit kommt mit $-3,85 \text{ s}$ **negativ** heraus — die Anpassung müsste die Antwort vor dem Sprung beginnen lassen. Es gibt also messbar einen Abfluss.
- PT1 und Torricelli liegen mit 3,42 mm und 3,90 mm dicht beieinander. Die Messung reicht nicht aus, um zwischen ihnen zu entscheiden — dazu müsste sie bis in die Sättigung laufen, wo sich die beiden Kurvenformen trennen. **Weitergerechnet wird mit PT1**, weil es den kleineren Restfehler hat und weil es geschlossen im Laplace-Bereich behandelt werden kann.
- Der Restfehler von 3,42 mm ist rund 55-mal so groß wie das Rauschen des Sensors (0,062 mm). Er ist also **kein Messrauschen, sondern Modellfehler**: die wirkliche Strecke ist nicht genau ein PT1. Das ist die ehrliche Genauigkeitsangabe des Modells.

5.4 Die Kennwerte des Behälters

Aus der PT1-Anpassung folgen unmittelbar:

Größe	Formel	Wert
Stellverstärkung	$K_u = K/100$	1,8748 mm je %
Zeitkonstante	T	36,69 s
Totzeit	T_d	1,252 s
Abflussbeiwert	$k_a/A = 1/T$	$0,02726 \text{ s}^{-1}$
Zuflussbeiwert	$k_p/A = K_u/T$	$0,05110 \text{ mm}/(\text{s} \cdot \%)$
Anfangssteigung bei 100 %	K/T	5,11 mm/s
Beharrungshöhe bei 100 %	$h_0 + K$	237,6 mm

Die Anfangssteigung ist die Probe von Hand: unmittelbar nach dem Sprung ist noch kein Abfluss aufgebaut, also ist $\dot{h} = K_u u/T = 1,8748 \cdot 100/36,69 = 5,11 \text{ mm/s}$. Im Bild der Sprungantwort ist das die Tangente durch den Knick bei T_d .

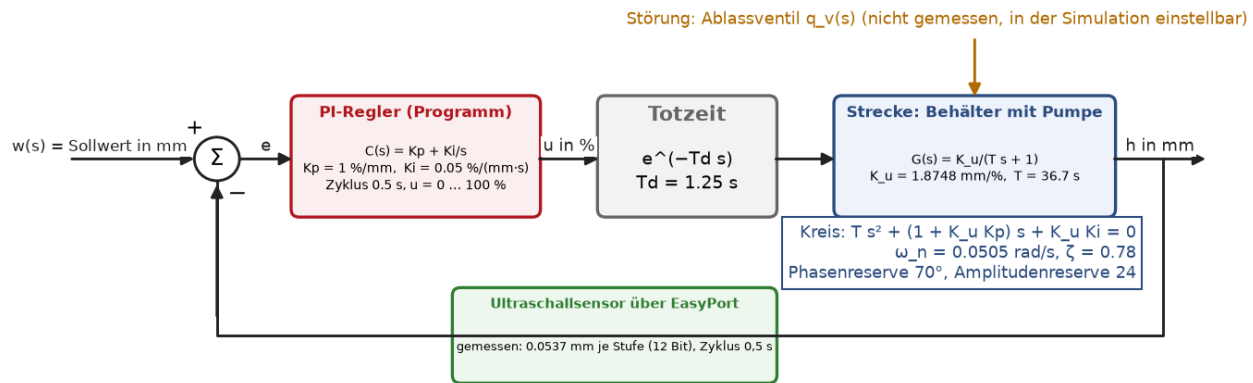
Nicht belegbar: Der lichte Querschnitt A ist nicht gemessen. Aus der Sprungantwort lassen sich nur die **Verhältnisse** k_a/A und k_p/A bestimmen; A , k_a und k_p einzeln erst, wenn der Durchfluss in l/min kalibriert vorliegt. Der Durchflusssensor an EW1.4 liefert dazu eine Spannung, deren Kennlinie im Programm nicht steht. Deshalb steht in diesem Blatt die Füllhöhe in Millimetern und nie ein Volumenstrom in Litern.

6 Der geschlossene Kreis

6.1 Übertragungsfunktionen

Strecke und Regler:

Der geschlossene Kreis im Laplace-Bereich (Strecke aus der Sprungantwort vom 25.06.2026)



parallel, nur Anzeige: Kamera → Bildauswertung → h_{Kamera} in mm (2-Punkt-Kalibrierung); die Regelung nutzt allein den Ultraschallsensor

Abbildung 4: Der geschlossene Kreis im Laplace-Bereich

$$G(s) = \frac{K_u}{T s + 1} e^{-T_d s}, \quad C(s) = K_p + \frac{K_i}{s} = \frac{K_p s + K_i}{s}.$$

Offener Kreis und Führungsübertragungsfunktion (zunächst ohne Totzeit):

$$L(s) = \frac{K_u (K_p s + K_i)}{s (T s + 1)}, \quad T_w(s) = \frac{L}{1 + L} = \frac{K_u (K_p s + K_i)}{T s^2 + (1 + K_u K_p) s + K_u K_i}.$$

Mit den Zahlen $K_u = 1,8748 \text{ mm/\%}$, $T = 36,69 \text{ s}$, $K_p = 1,0 \text{ \%/mm}$, $K_i = 0,05 \text{ \%/ (mm·s)}$:

$$36,69 s^2 + 2,8748 s + 0,09374 = 0$$

Das ist die Kennkreisgleichung. Alle drei Beiwerte sind auf einer Zeile nachzurechnen: $1 + K_u K_p = 1 + 1,8748 = 2,8748$ und $K_u K_i = 1,8748 \cdot 0,05 = 0,09374$.

6.2 Kennwerte und Stabilität

Größe	Formel	Wert
Kennkreisfrequenz	$\omega_n = \sqrt{\frac{K_u K_i}{T}}$	0,05055 rad/s
Dämpfung	$\zeta = \frac{1 + K_u K_p}{2 \sqrt{T K_u K_i}}$	0,775
Abklingrate	$\sigma = \frac{2T}{1 + K_u K_p}$	0,03918 s ⁻¹
gedämpfte Kreisfrequenz	$\omega_d = \omega_n \sqrt{1 - \zeta^2}$	0,03194 rad/s
Pole	$-\sigma \pm j \omega_d$	-0,03918 ± j 0,03194
Schwingungsdauer	$2\pi / \omega_d$	196,7 s
Durchtrittsfrequenz	$ L(j\omega) = 1$	0,06045 rad/s
Phasenreserve	$180^\circ + \arg L$	70,3°
Amplitudenreserve	$1/ L $ bei $\arg L = -180^\circ$	24,3

Beide Pole liegen links der imaginären Achse: **der Kreis ist stabil**. Die Dämpfung 0,775 liegt nahe am üblichen Entwurfswert $1/\sqrt{2} = 0,707$; die Phasenreserve von 70° ist reichlich.

Die Totzeit ist in diesen Zahlen noch nicht enthalten. Setzt man sie als Padé-Näherung erster Ordnung an, $e^{-T_d s} \approx \frac{1 - T_d s/2}{1 + T_d s/2}$, so wird die Kennkreisgleichung kubisch, und ihre Wurzeln sind

$$s_1 = -1,494 \text{ s}^{-1}, \quad s_{2,3} = -0,04014 \pm j 0,03350 \text{ s}^{-1}.$$

Alle drei liegen links. Die Totzeit verschiebt das dominierende Polpaar nur geringfügig — das war zu erwarten: $T_d = 1,25 \text{ s}$ ist gegen die Schwingungsdauer von 197 s verschwindend. Auch die Abtastung mit $0,5 \text{ s}$ ist gegenüber dieser Schwingungsdauer ohne Bedeutung (Verhältnis $1 : 394$), weshalb die zeitkontinuierliche Rechnung zulässig ist.

6.3 Störübertragungsfunktion

Das geöffnete Ablassventil wirkt als zusätzlicher Abfluss q_v (in mm/s an der Füllhöhe gemessen). In der Bilanz steht er neben dem Zufluss:

$$T \dot{h} = K_u u - (h - h_0) - T q_v.$$

Mit $\Delta u = -C(s) \Delta h$ folgt

$$\frac{\Delta h(s)}{q_v(s)} = \frac{-T s}{T s^2 + (1 + K_u K_p) s + K_u K_i}.$$

Für $s \rightarrow 0$ geht das gegen null: **eine dauerhafte Störung wird vollständig ausgeregelt**. Das ist die Wirkung des I-Anteils und braucht keinen Zahlenwert. Was einen Zahlenwert braucht, ist der Einbruch unterwegs (nächster Abschnitt).

6.4 Zwei unabhängige Wege zu denselben Kennwerten

Das ist Grundsatz 1 der Werkstatt, hier ausgeführt:

- **Weg A — Schritt für Schritt** (Modell/streckenmodell.py, Funktion simuliere): der Kreis wird mit genau dem Algorithmus des Programms gerechnet — Zyklus $0,5 \text{ s}$, Totzeit über einen Verzögerungspuffer, Anti-Windup, 16-Bit-Quantisierung von Messwert und Stellgröße, Strecke mit dem Euler-Verfahren bei $0,1 \text{ s}$.
- **Weg B — geschlossen** (Modell/nachrechnung.py): Laplace, ohne Abtastung, ohne Totzeit, ohne Stellgrößenbegrenzung. Die Formeln stehen unten und sind auf Papier nachvollziehbar.

Arbeitspunkt bei 150 mm. Im Beharrungszustand fließt so viel zu wie ab, also $K_u u = h - h_0$:

$$u = \frac{150 - 50,16}{1,8748} = 53,254 \text{ } \%$$

Weg A liefert $53,254 \text{ } \%$. **Unterschied: 0,000 Prozentpunkte.**

Sollsprung 150 → 200 mm. Weg B: aus $y(0) = 0$ und $y'(0) = \Delta w g$ mit $g = K_u K_p / T$ folgt

$$\frac{y(t)}{\Delta w} = 1 - e^{-\sigma t} (\cos \omega_d t - B \sin \omega_d t), \quad B = \frac{g - \sigma}{\omega_d} = 0,37327.$$

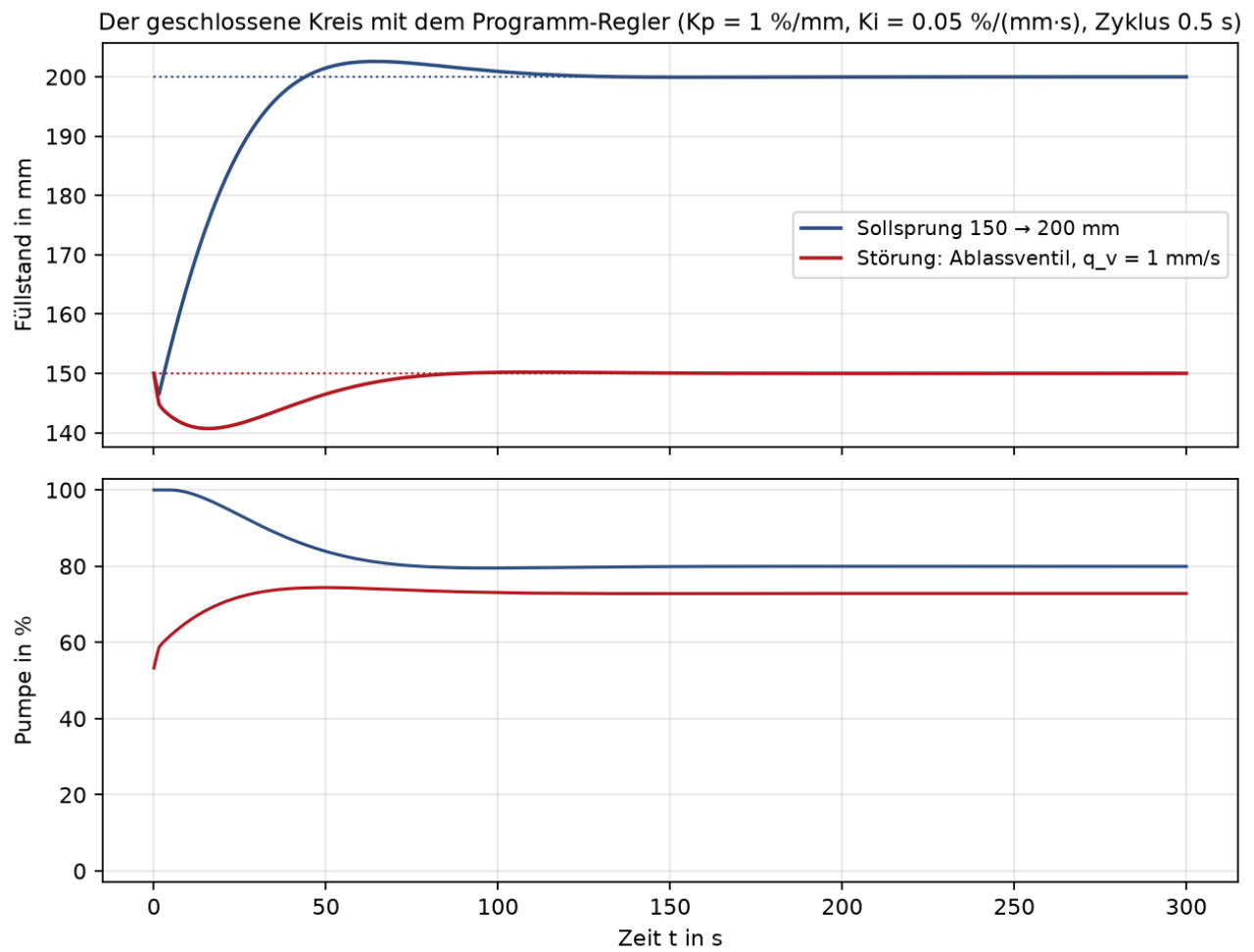


Abbildung 5: Der geschlossene Kreis mit dem Programm-Regler

Das Maximum liegt bei $t = 59,4$ s und beträgt 106,59 % — also **3,29 mm** Überschwingen bei 50 mm Sprunghöhe. Weg A liefert **2,62 mm** bei rund 60 s. Der Unterschied von 0,67 mm hat einen Namen: **die Stellgrößenbegrenzung**. Weg A fährt die Pumpe zu Beginn in den Anschlag bei 100 % (im Bild gut zu sehen), Weg B kennt keinen Anschlag und treibt darum weiter. Die bleibende Abweichung ist auf beiden Wegen null (Weg A: 0,004 mm, das ist die Quantisierung); die Einschwingzeit in ein 2-mm-Band beträgt 81 s.

Störsprung, Ablassventil $q_v = 1$ mm/s. Weg B: mit $\Delta h(s) = -q_v/(s^2 + 2\sigma s + \omega_n^2)$ folgt $\Delta h(t) = -\frac{q_v}{\omega_d} e^{-\sigma t} \sin \omega_d t$, größter Betrag bei $\tan(\omega_d t) = \omega_d/\sigma$, also bei $t = 21,4$ s mit **8,55 mm**. Weg A liefert **9,30 mm** Einbruch und regelt in 60 s aus. Der Unterschied von 0,75 mm (8 %) ist die Wirkung von Totzeit (1,25 s) und Abtastung (0,5 s): der Regler greift später ein.

Beide Unterschiede haben also eine benennbare Ursache, und beide Wege bestätigen einander in Größenordnung, Vorzeichen und Zeitpunkt. Was sie **nicht** ersetzen, ist die Messung am geschlossenen Kreis: die liegt nicht vor (siehe „Was offen ist“).

7 Bedienung

7.1 Das Programm am PC

1. EasyPort anschließen, Anlage einschalten, Kamera mit Strom versorgen.
2. Wasserstandsregelung_Bild_6.py starten. Links erscheinen Reglerfelder, Messwerte und die beiden Verläufe; rechts das Kamerabild mit seinen Bedienfeldern.
3. **Sollwert, Kp, Ki** eintragen (Voreinstellung 150 mm, 1,0, 0,05).
4. **„Regler EIN“** drücken. Der Knopf zeigt danach den Zustand an. „Regler AUS“ hält die Pumpe an.
5. ****„Ventil ÖFFNEN (Störung)“** öffnet das Ablassventil; der Knopf wird rot und meldet „STÖRUNG AKTIV“. Erneutes Drücken schließt es.
6. „Integral zurücksetzen“ nullt den I-Anteil, „Pumpe STOP“ hält die Pumpe unabhängig vom Regler an.

7.2 Die Kamera einstellen

1. Der **ROI** wird mit gedrückter Maustaste über dem Kamerabild aufgezogen (grüner Rahmen). Er soll den **gesamten Glasinhalt** umfassen — Luft oben, Wasser unten —, aber keinen Tisch.
2. Die **Methode** wählen: „Helligkeit“ für trübes Wasser (Voreinstellung), „Canny“ für eine deutliche Kante, „Farbfilter (HSV)“ für gefärbtes Wasser. Nur die Schieber der gewählten Methode sind sichtbar.
3. Die **Empfindlichkeit** so weit senken, bis die Linie stabil steht; die **Konfidenzschwelle** niedriger machen heißt stabiler, aber auch leichtgläubiger; **Hold** bestimmt, wie viele Bilder lang ein alter Wert stehenbleibt.
4. **Weißabgleich** (Knopf „WB“) durchschalten, wenn sich die Farben mit der Beleuchtung ändern.

7.3 Kalibrieren

1. Füllstand auf einen bekannten Wert bringen (am Ultraschallwert ablesen), diesen Wert in „Füllstand jetzt [mm]“ eintragen, **„Punkt setzen“**.
2. Füllstand auf einen deutlich anderen Wert bringen, eintragen, **„Punkt setzen“**.
3. Die Zeile darunter meldet „Kalibriert“ (im Programm mit einem Haken davor) und beide Punkte. Ab jetzt zeigt die Kamera Millimeter.
4. „Löschen“ verwirft die Kalibrierung.

Je weiter die beiden Punkte auseinanderliegen, desto kleiner wirkt sich ein Fehler in der Erkennung auf die Steigung der Geraden aus — man nehme sie nahe an den Rändern des Arbeitsbereichs.

7.4 Aufzeichnen

„Screenshot“ legt ein JPEG mit eingezeichneter Linie im Ordner ESP32_Aufnahmen ab, „Video“ schreibt eine AVI-Datei (XVID, 24 Bilder/s, 480 × 360) mit allen Einblendungen, bis erneut gedrückt wird.

8 Die Seite im Browser

Zum Manuskript gehört eine eigenständige HTML-Seite, Seite/Wasserstand_1.0.html, erzeugt von Seite/erstelle_wasserstand_seite.py. Sie läuft **offline**, ohne Anlage, ohne Kamera und ohne Netzzugang, und enthält:

- einen **gezeichneten Behälter** als Nachbildung des Kamerabildes (480 × 360), mit Schiebern für Beleuchtung, Rauschen und Spiegelung;
- **dieselbe Bildauswertung**, Zeile für Zeile aus dem Python-Programm nach JavaScript übertragen (Graustufen mit derselben Gewichtung, Gauß (1 × 31), Zeilenmittel, Mittel über 5, Gradient, Suche im oberen 85-%-Bereich, Schwelle −0,3· Empfindlichkeit, Konfidenz, Halten, Mittel über 12, Zweipunktkalibrierung);
- **denselben Regler** (PI mit Anti-Windup, Zyklus 0,5 s, Integral-Reset bei Sollsprung über 1 mm);
- **dieselbe Strecke** ($T\dot{h} = K_u u(t - T_d) - (h - h_0) - Tq_v$ mit den gemessenen Kennwerten);
- Schieber für den Sollwert, einen Knopf für die Störung (Ablassventil) und die **laufenden Verläufe** $h(t)$ und $u(t)$;
- dieses Manuskript als Reiter „Dokumentation“.

Der erste Bildschirm ist bedienbar: Behälter, Schieber und Knöpfe stehen oben, der Text dahinter.

Die Seite ist gleichzeitig der **dritte** Rechenweg: sie führt den Kreis in einer anderen Sprache und mit einem anderen Zeitgeber aus. Ihr Prüfprogramm Seite/pruefe_seite.mjs fährt sie in einem echten Chromium, misst Erkennungsfehler, Überschwingen, bleibende Abweichung und Störausregelung und vergleicht sie mit den Schranken in Seite/PRUEFPLAN_Seite.md. Was dabei herauskommt (18 Kriterien, keine Beanstandung):

Größe	Seite (JavaScript)	Python	Schranke
Bildauswertung am Prüfbild: gefundene Zeile	109	109	gleich
dieselbe, Konfidenz Erkennung gegen den gezeichneten Stand (100 ... 255 mm)	0,2092	0,2077	Unterschied ≤ 0,005
Sollsprung 150 → 200 mm: Überschwingen	größte Abweichung 0,78 mm	—	≤ 3,0 mm
dieselbe, bleibende Abweichung	2,62 mm	2,62 mm	Unterschied ≤ 0,2
dieselbe, Einschwingzeit (±2 mm)	0,002 mm	0,004 mm	≤ 0,1 mm
Störung 1 mm/s: größter Einbruch	81 s	81 s	Unterschied ≤ 0,2
	9,3 mm	9,3 mm	Unterschied ≤ 0,2

Größe	Seite (JavaScript)	Python	Schranke
dieselbe, ausgeregelt in	60 s	60 s	Unterschied $\leq 0,2$

Der Unterschied in der Konfidenz von 0,0014 hat eine benennbare Ursache: OpenCV rundet im Weichzeichner jeden Bildpunkt auf ganze Graustufen, die Seite mittelt zuerst über die Zeile und glättet dann (beides linear, also vertauschbar) und rundet dabei nicht. Die bleibende Abweichung von 0,002 gegen 0,004 mm ist die Quantisierung des Sensorwerts, deren Stufe 0,0067 mm beträgt.

9 Was offen ist

Was hier nicht steht, steht nicht, weil es nicht belegt ist. Diese Liste gehört zum Ergebnis:

1. **Es gibt keine Messung am geschlossenen Kreis.** Vorliegend ist allein die Sprungantwort der Strecke vom 25.06.2026. Alle Aussagen über Überspringen, Einschwingzeit und Störausregelung sind **gerechnet**, nicht gemessen. Zwei unabhängige Rechenwege ersetzen keine dritte Messung.
2. **Der Abfluss q_v des Ablassventils ist nicht gemessen.** Der Wert $q_v = 1$ mm/s ist eine gewählte Größe für die Referenzsimulation, keine Eigenschaft der Anlage. Solange er nicht gemessen ist, sagt der Einbruch von 9,3 mm nichts über den wirklichen Vorgang.
3. **Der lichte Querschnitt A des Behälters ist nicht gemessen**, ebenso wenig die Pumpenkennlinie. Darum stehen hier nur die Verhältnisse k_a/A und k_p/A und keine Volumenströme.
4. **Die Kennlinien von Durchfluss- und Drucksensor** stehen nicht im Programm; die beiden Werte werden als Spannung angezeigt und sonst nicht verwendet.
5. **Die Kameraoptik ist nicht vermessen** (Abstand, Bildwinkel, Blickrichtung, Verzeichnung). Die Gerade Bildpunkt $\rightarrow$ Millimeter ist eine Annahme, die durch die Zweipunktkalibrierung nur an zwei Stellen gestützt wird. Ob sie dazwischen gilt, ist nicht geprüft.
6. **PT1 und Torricelli sind an dieser Messung nicht unterscheidbar** (3,42 mm gegen 3,90 mm Restfehler). Zur Entscheidung bräuchte es eine Sprungantwort bis in die Sättigung.
7. **Der Endwert der Strecke ist hochgerechnet**, nicht gemessen: die Messung deckt 1,21 Zeitkonstanten ab. $K = 187,5 \pm 3,8$ mm.
8. **Die Genauigkeit des Ultraschallsensors ist nicht bestimmt.** Bekannt ist nur die Stufe des Wandlers (0,054 mm) und das Rauschen im Stillstand (0,062 mm).
9. **Vom Aufbau liegt kein Lichtbild vor.** Die Datei Wasserstand.jpg zeigt ein Bildschirmfoto der Programmoberfläche mit noch schwarzem Kamerafeld, nicht die Anlage. Sie ist als solches beschriftet und wird nicht als Aufbaubild ausgegeben.
10. **Die Bildauswertung ist nicht an echten Kamerabildern vermessen.** Alle Zahlen zur Erkennung stammen von gezeichneten Rechenbildern. Damit ist gezeigt, dass das Verfahren rechnet, was es rechnen soll — nicht, wie genau es an dieser Anlage arbeitet.

10 Anhang: die Dateien

Datei	Inhalt
Wasserstandsregelung_Bild_6.py	das Programm am PC: EasyPort, PI-Regler, Bildauswertung, Oberfläche
Daten/sprungantwort_2026-06-25.csv	die Messreihe: Zeit in s, Füllhöhe in mm, 370 Punkte

Datei	Inhalt
Modell/streckenmodell.py	Anpassung der drei Modelle, Kreisrechnung, Schritt-für-Schritt-Simulation → streckenmodell.json
Modell/nachrechnung.py	der zweite, geschlossene Weg → nachrechnung.json
Modell/bv_referenz.py	die Bildauswertung, wörtlich aus dem Programm herausgeschnitten
Modell/erstelle_bv_referenz.py	schneidet sie heraus (nicht von Hand ändern)
Bilder/erstelle_bilder.py	erzeugt die drei gezeichneten Bilder dieses Blattes
Seite/erstelle_wasserstand_seite.py	erzeugt die HTML-Seite (Fassung aus Seite/VERSION)
Seite/pruefe_seite.mjs	prüft die Seite im echten Browser → pruefe_seite.json
Seite/PRUEFPLAN_Seite.md	der Prüfplan mit den Schranken

Alle Bilder liegen in Bilder/ und werden von den genannten Programmen erzeugt; von Hand ist keines gezeichnet.